

Perceptron Learning for AND Gate — From Scratch

1. The Dataset

x ₁	x ₂	y
0	0	0
0	1	0
1	0	0
1	1	1

We want the perceptron to **learn** this AND pattern purely from data — starting with zero knowledge.

2. The Perceptron Model

Weighted sum:

$$z = w_1x_1 + w_2x_2 + b$$

Step (activation) function:

$$\hat{y} = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

Weight update rule (applied *only* when the prediction is wrong):

$$w_1 \leftarrow w_1 + \eta(y - \hat{y})x_1$$

$$w_2 \leftarrow w_2 + \eta(y - \hat{y})x_2$$

$$b \leftarrow b + \eta(y - \hat{y})$$

We'll use learning rate $\eta = 1$.

Initial values:

$$w_1 = 0, \quad w_2 = 0, \quad b = 0$$

3. Epoch 1 — Going through all 4 rows once

Row 1: $x_1 = 0, x_2 = 0, y = 0$

Compute z:

$$z = w_1x_1 + w_2x_2 + b = (0)(0) + (0)(0) + 0 = 0$$

Predict:

$$z = 0 \geq 0 \Rightarrow \hat{y} = 1$$

Compare: actual $y = 0$, predicted $\hat{y} = 1 \rightarrow$ **WRONG**

$$\text{error} = y - \hat{y} = 0 - 1 = -1$$

Update weights (multiply error by each input):

$$w_1 = 0 + (1)(-1)(0) = 0$$

$$w_2 = 0 + (1)(-1)(0) = 0$$

$$b = 0 + (1)(-1) = -1$$

(w_1 and w_2 don't move because $x_1 = 0, x_2 = 0$ — anything times 0 is 0. Only the bias, which has no input attached, shifts.)

Weights now: $w_1 = 0, w_2 = 0, b = -1$

Row 2: $x_1 = 0, x_2 = 1, y = 0$

Compute z:

$$z = (0)(0) + (0)(1) + (-1) = -1$$

Predict:

$$z = -1 < 0 \Rightarrow \hat{y} = 0$$

Compare: actual $y = 0$, predicted $\hat{y} = 0 \rightarrow$ **CORRECT**

$$\text{error} = 0 - 0 = 0$$

No update needed — weights stay: $w_1 = 0, w_2 = 0, b = -1$

Row 3: $x_1 = 1, x_2 = 0, y = 0$

Compute z:

$$z = (0)(1) + (0)(0) + (-1) = -1$$

Predict:

$$z = -1 < 0 \Rightarrow \hat{y} = 0$$

Compare: actual $y = 0$, predicted $\hat{y} = 0 \rightarrow$ **CORRECT**

No update needed — weights stay: $w_1 = 0, w_2 = 0, b = -1$

Row 4: $x_1 = 1, x_2 = 1, y = 1$

Compute z:

$$z = (0)(1) + (0)(1) + (-1) = -1$$

Predict:

$$z = -1 < 0 \Rightarrow \hat{y} = 0$$

Compare: actual $y = 1$, predicted $\hat{y} = 0 \rightarrow$ **WRONG**

$$\text{error} = y - \hat{y} = 1 - 0 = 1$$

Update weights:

$$w_1 = 0 + (1)(1)(1) = 1$$

$$w_2 = 0 + (1)(1)(1) = 1$$

$$b = -1 + (1)(1) = 0$$

Weights now: $w_1 = 1, w_2 = 1, b = 0$

4. Summary of Epoch 1

Row	x_1	x_2	y	z	$\hat{y}$	Result	w_1	w_2	b
start	—	—	—	—	—	—	0	0	0
1	0	0	0	0	1	WRONG	0	0	-1
2	0	1	0	-1	0	CORRECT	0	0	-1
3	1	0	0	-1	0	CORRECT	0	0	-1
4	1	1	1	-1	0	WRONG	1	1	0

End of Epoch 1: $w_1 = 1, w_2 = 1, b = 0$

5. Testing after Epoch 1 — not fully learned yet

x ₁	x ₂	z = x ₁ +x ₂ +0	Prediction	Actual y	Correct?
0	0	0	1	0	WRONG
0	1	1	1	0	WRONG
1	0	1	1	0	WRONG
1	1	2	1	1	CORRECT

Only 1 out of 4 correct — one epoch clearly isn't enough. The perceptron must repeat this same process over **more epochs**, adjusting weights each time it's wrong, until it converges.

6. After training converges (multiple epochs later)

$$w_1 = 1, w_2 = 1, b = -1.5$$

x ₁	x ₂	z = x ₁ +x ₂ -1.5	Prediction	Actual y	Correct?
0	0	-1.5	0	0	CORRECT
0	1	-0.5	0	0	CORRECT
1	0	-0.5	0	0	CORRECT
1	1	0.5	1	1	CORRECT

All 4 correct — the perceptron has now learned the AND function.

7. OR Gate — Dataset & Converged Solution

Dataset:

x ₁	x ₂	y
0	0	0
0	1	1
1	0	1
1	1	1

The OR gate only needs **one** input to be 1 for the output to be 1 — so the decision boundary sits much closer to the origin than AND's.

A converged solution:

$$w_1 = 1, w_2 = 1, b = -0.5$$

x ₁	x ₂	$z = x_1 + x_2 - 0.5$	Prediction	Actual y	Correct?
0	0	-0.5	0	0	CORRECT
0	1	0.5	1	1	CORRECT
1	0	0.5	1	1	CORRECT
1	1	1.5	1	1	CORRECT

All 4 correct — the perceptron has learned the OR function.

Note the pattern: compared to AND ($b = -1.5$), OR only needed to shift the bias to $b = -0.5$ — the boundary line moved closer to the origin so that even a single active input crosses it.

8. NAND Gate — Dataset & Converged Solution

Dataset:

x ₁	x ₂	y
0	0	1
0	1	1
1	0	1
1	1	0

NAND is simply AND with the output flipped — so its solution is the AND solution with every weight and bias **negated**.

A converged solution:

$$w_1 = -1, w_2 = -1, b = 1.5$$

x ₁	x ₂	$z = -x_1 - x_2 + 1.5$	Prediction	Actual y	Correct?
0	0	1.5	1	1	CORRECT
0	1	0.5	1	1	CORRECT
1	0	0.5	1	1	CORRECT
1	1	-0.5	0	0	CORRECT

All 4 correct — the perceptron has learned the NAND function.

9. Quick Reference — All Four Gates

Gate	w_1	w_2	b	Boundary line
AND	1	1	-1.5	$x_1 + x_2 = 1.5$
OR	1	1	-0.5	$x_1 + x_2 = 0.5$
NAND	-1	-1	1.5	$x_1 + x_2 = 1.5$ (flipped side)
XOR	—	—	—	Not linearly separable — a single perceptron cannot learn XOR

XOR is included here as a reminder: no straight line can separate XOR's outputs, which is why a single-layer perceptron fails on it and a hidden layer (multi-layer perceptron) is required.